Reducing the cost of quantum-kernel predictions

Interstate Quantum Hackathon for Beginners — Fall 2026

Group: Kieran McCrorey (solo)

Participant: Kieran Edward McCrorey

Affiliation: University of Alabama in Huntsville

Department: Electrical and Computer Engineering

Career level: Undergraduate, electrical engineering

Submission category: Rising-star

1. Opening perspective and AI disclosure

I just want to start by saying thank you to the organizers of this event. This has been such a fun and informative experience. I joined this hackathon with absolutely zero quantum experience. Now I can see what the field actually looks like and where this technology actually might be headed rather than a detached perspective only built through headlines and hype. I also want to be extremely forthcoming about my AI usage in assisting me with this project. All of the code and technical details in the notebook were created with the assistance of Claude Code and my Hermes agent. I started out by trying to follow along with the math and technical details, but that ended up not being where I focused most of my attention. I don't think it's bold to say that people in the field of Quantum are right now using AI to do their work. I certainly will be when I enter the workforce. So, I wanted to simulate that process more than anything. I haven't outsourced the learning process to AI though, I spent a lot of time understanding the experiments

themselves and choosing which things to optimize. This report is my own words, not AI written so I can faithfully demonstrate what I learned and what I enjoyed about the process.

2. Summary

- I investigated how to reduce the execution cost of predictions from a quantum-kernel classifier while preserving prediction quality.
- We tested adaptive stopping, shot allocation, support-vector skipping, circuit compilation, qubit selection, noise correction, and direct measurement of the classifier's decision value.
- On 814 held-out inputs in ideal shot-noise simulation, direct measurement with adaptive stopping averaged about 65 shots per prediction, compared with 20,480 for the strict fixed-shot baseline. True-label accuracy was approximately 99.74% versus 99.76%, respectively. This is about 315 times fewer shots, not a measured runtime speedup.
- On twenty inputs executed on IBM's ibm_fez, the fastest tested configuration used 5 reported QPU seconds versus 110 for the strict baseline. Every tested configuration matched all twenty ideal reference predictions and classified nineteen true labels correctly.
- The fewest-shot hardware configuration was not the fastest: adaptive direct measurement used 7,616 shots across five jobs and consumed 10 QPU seconds; fixed direct measurement used 40,960 shots in one job and consumed 5 seconds.
- AI handled much of the implementation, literature research, and mathematical analysis. I directed the investigation and worked through the meaning of the experiments and their limitations.
-

Keep shot savings and measured runtime savings separate.

Why I chose QML

I chose QML because it sounded like the most insane application on that list. Machine learning seemed like a miracle a few weeks ago, but learning about the underlying mathematics makes it seem much more real. Optimization was my favorite part of the process, because that was the most tangible result. I only have so much QPU time, so minimizing time allows me to see how the problem I was working on has real world value. Even if QML can't beat classical computers yet, making it more efficient brings that closer to reality, which is so cool to me.

How my question evolved

The question I started with was: "Can we use fewer shots without losing prediction quality?" Fewer shots in theory means less compute, and thus a faster and more efficient result. I thought this aspect alone would be enough work for the whole project, but it was tested and answered pretty quickly. Then I broadened the question to: "What is the cheapest trustworthy prediction?" Shot count is only one of many factors that can be improved, so I wanted to investigate other methods for improving efficiency while still maintaining accuracy. This is where I brought in simulated noise to test our methods in a more realistic environment, and eventually the feasibility on real hardware.

Workflow

My workflow was mostly focused on asking targeted questions, having AI investigate possible solutions, testing those configurations in Quiskit, and reviewing the data to figure out what to research next. I had my main agents researching and explaining concepts to me, while they

directed subagents to carry out bounded tests on their research. The main agent would then read the subagent's report, test their code, and report back to me.

The first optimization: deciding when to stop measuring

Our first method for reducing the shots was to only use more shots on uncertain points closer to the boundary, up until an optimal cap. More shots produce more accurate results, but take more shots obviously. Reducing them is a good thing for efficiency until the accuracy takes a hit. We tested accuracy vs. the number of shots to determine where to put this cap for the adaptive shots.

The direct-measurement approach and proof

So things got a little complicated here and my technical explanation for sure lacks some depth, but the shots previously were estimating 20 similarities between examples in the graph separately. Then, my Hermes agent created a proof for a method to measure the weighted combination of those 20 similarities simultaneously. This reduced our compute cost by a factor of 20, and was the biggest optimization we applied to this problem.

Results

The most interesting part of the real hardware simulation was seeing how the number of shots didn't directly correlate with QPU time. The adaptive result had the least shots, but took longer than the run with the direct comparison result with 4x the shots. That was due to the QPU usage being split into 5 jobs, and IBM's system used time in between jobs as well. Each job may have taken less time, but it still took longer.

What I learned

This was a wonderful process. I've worked on many side projects with my AI Agents and learned a thing or two about how to use AI to solve problems, but this is the first real problem I've solved with AI. The technical aspects being handled by AI allowed me to focus more on the application of the technology and the broader process of optimization and testing. I learned how to direct these resources towards a task in a fixed amount of time, while trying to learn along the way. I've learned a lot about quantum computing obviously, from the lectures and the keynote, and my own experience with the problem. This is definitely a hard problem to tackle, especially with the not-obvious connection between the factors. I initially was sure that an adaptive system would save a lot more QPU time by reducing the number of shots, but I was wrong for a tangentially related reason. I would've liked to explore more options for optimization since the quantum hardware gave us so much data, but I ran out of time. I think I had a successful exploration though, and I learned more than I thought I would by outsourcing the technical stuff.